

Uber

Functional requirements:

- user should be able to book a cab based on his preference,
- user should be able to search for locations based on his preference,
- rider should be able to accept/reject ride

Non-functional requirements;

- Data-partitioning,
- load balancing,
- fault tolerance,
- vertical scalability,
- horizontal scalability,
- low latency,
- high throughput,

CAP theorem (Here we need to prioritise Strong Consistency for booking scenario in order to avoid double booking/double payment & for search scenario we need to prioritise 99.99% uptime i,e Availability>>Consistency)

Core Entities:

- user,
- book,
- payment,
- history

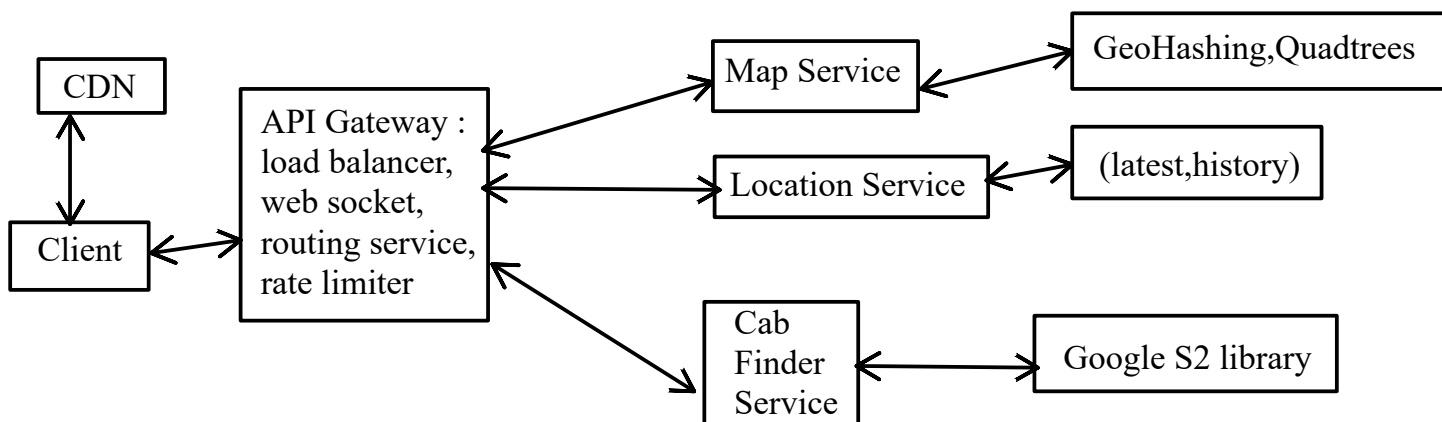
API :

POST:/booking

```
{
  userID,
  bookingID.
  bookingStatus
}
```

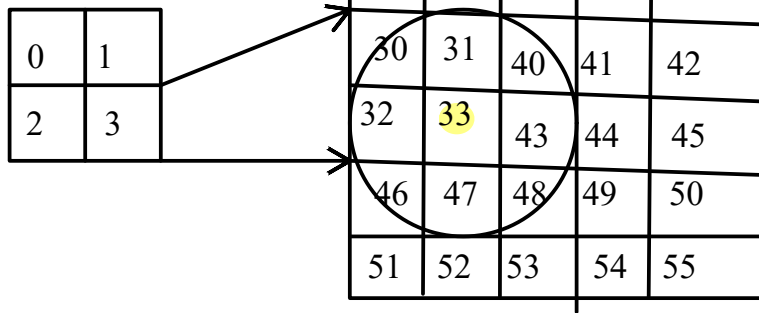
GET:/search/{locationID}

HLD :

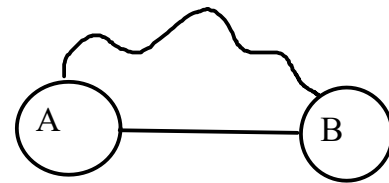


GeoHashing, Quadtrees

(lat,long)->chunkID



ChunkID->(riders)



We cannot use Euclidian distance ,
because path between two points will
not be straight line ,in that scenaio
we need to use Google S2 library

Through consistent hashing , gossip
Protocol
all the nodes will get the updated value